

安徽大学 2024-2025 学年第二学期
《FPGA 数字系统设计》考试试卷 (A 卷)

标准答案与解析

一、单项选择题 (共 10 分, 每题 1 分)

1. 答案: C

解析: 连续赋值语句 (`assign`) 只能用于组合逻辑设计, 不能用于时序逻辑设计。时序逻辑设计必须在 `always` 块中使用非阻塞赋值 (`<=`)。

2. 答案: D

解析: A、B、C 三种端口声明在 Verilog-2001 及以上标准中均是完全合法的。

3. 答案: C

解析: `initial` 语句仅在仿真开始时执行一次, 且不可综合, 因此它并不能在实际硬件电路上电时自动反复执行或指导硬件行为。

4. 答案: D

解析: 条件表达式 `y = (a > b) ? a : b;` 的含义是: 若 `a > b` 成立则输出 `a`, 否则输出 `b`。因此其功能是输出 `a` 和 `b` 中的较大值。

5. 答案: C

解析: `case` 语句可以嵌套使用。A 错误, `default` 不是必须的 (虽然为了防止生成锁存器建议加上); B 错误, 分支条件如果重叠会按照优先级执行, 但通常建议互斥; D 错误, `case` 也可用于时序逻辑。

6. 答案: C

解析: Verilog 的标识符可以由字母、数字、下划线和 `$` 组成, 但 `**` 不能以数字开头 `**`。因此 `3state` 是非法的。

7. 答案: D

解析: 在同一个 `always` 块中, `**` 严禁 `**` 混合使用阻塞赋值 (`=`) 和非阻塞赋值 (`<=`), 这会导致不可预测的仿真与综合行为。

8. 答案: B

解析: `always` 语句既可以用于时序逻辑 (如带时钟触发), 也可以用于组合逻辑 (如 `always @(*)`)。

9. 答案: C

解析: ``timescale 10ns/100ps` 中, 斜杠前面的 `10ns` 是时间单位, 斜杠后面的 `100ps` 是时间精度。

10. 答案: A

解析: 这是标准的显式端口例化 (名称映射) 语法: `.port_name(signal_name)`。

二、填空题 (共 15 分, 每空 1 分)

11. 寄存器数据类型 (`reg`)、线网数据类型 (`net/wire`)

12. `.xdc` (Xilinx Design Constraints)

13. `a+b+c+d / e`

14. B

15. 现场可编程门阵列

16. 组合

17. PS (Processing System, 处理系统)、PL (Programmable Logic, 可编程逻辑)

18. Moore (摩尔) 型、Mealy (米里) 型

19. 独热码 (One-hot)、二进制码 (Binary)

20. `8'b11001100`

三、简答题 (共 35 分)

21. 简述 FPGA 的基本组成结构, 并说明各部分的作用。(10 分)

答: FPGA 的基本组成结构主要由以下三部分组成:

1. 可编程输入/输出单元 (IOB, Input/Output Block): 作为芯片内部逻辑与外部引脚之间的接口, 负责不同电气特性下的信号驱动与匹配、电气互连、阻抗匹配等。
2. 可编程逻辑块 (CLB, Configurable Logic Block): 是 FPGA 实现逻辑功能的核心单元。每个 CLB 通常包含查找表 (LUT)、触发器 (Flip-Flop) 和多路复用器。其中 LUT 用于实现任意组合逻辑, 触发器用于实现时序逻辑。
3. 可编程布线资源 (Routing Resources): 分布在芯片内部的各种长度的金属连线及可编程开关阵列, 负责将 IOB 和 CLB 等内部资源按设计要求柔性地连接起来, 构成复杂的数字系统。

(注: 现代 FPGA 还包含嵌入式块状 RAM、数字信号处理单元 DSP、时钟管理单元 DCM/PLL 等扩展结构。)

22. 分析 Verilog 中 wire 和 reg 型变量的区别, 并说明其使用场景。(10 分)

答:

1. 本质区别:

- wire 型变量表示硬件中的物理连线, 它本身不存储电荷或数据, 其电平值由驱动源 (如 assign 语句或门电路输出) 决定。如果断开驱动, 其值通常为高阻态 (bz)。
- reg 型变量代表具有保持功能的存储单元, 在仿真中它能保持上一次被赋予的值, 直到下一个赋值语句给它赋新值。在综合时, 它可能被映射为触发器 (时序逻辑) 或连线/锁存器 (组合逻辑)。

2. 使用场景语法限制:

- wire 型必须用 assign 连续赋值语句驱动, 或者作为模块例化的输出连接。
- reg 型变量必须在过程块 (如 always 块或 initial 块) 内部被赋值。

23. 请根据以下内容, 完成下面的运算 (用基数法二进制表示) (15 分)

- | | | |
|-------------|--------------|--------------|
| (1) 4'b1001 | (6) 4'b0011 | (11) 4'b1100 |
| (2) 4'b1000 | (7) 4'b1101 | (12) 4'b1111 |
| (3) 4'b1111 | (8) 4'b1111 | (13) 4'b1010 |
| (4) 4'b1001 | (9) 4'b1010 | (14) 4'b1000 |
| (5) 4'b1010 | (10) 4'b1110 | (15) 4'b0000 |

答:

四、程序设计题 (本大题共 40 分)

24. 根据输入信号 a, b 的大小关系, 求解两个数的差值模块 (10 分)

```
1 module data_minus(  
2     input clk,  
3     input rst_n,  
4     input [7:0] a,  
5     input [7:0] b,  
6     output reg [8:0] c  
7 );  
8  
9     always @(posedge clk or negedge rst_n) begin  
10         if (!rst_n) begin  
11             c <= 9'd0;  
12         end  
13         else begin  
14             if (a > b)  
15                 c <= {1'b0, a} - {1'b0, b};  
16             else  
17                 c <= {1'b0, b} - {1'b0, a};  
18         end  
19     end  
20  
21 endmodule
```

25. 利用有限状态机设计一个流水灯控制器 (15 分)

```
1 module led_fsm(  
2     input clk,  
3     input rst,  
4     output reg [3:0] led  
5 );  
6     // 设时钟 1MHz, 1 秒需要计数 1000_000 次  
7     reg [19:0] cnt;  
8     wire timer_1s = (cnt == 20'd999999);  
9  
10    // 1s 计数器  
11    always @(posedge clk or posedge rst) begin  
12        if (rst) cnt <= 20'd0;  
13        else if (timer_1s) cnt <= 20'd0;  
14        else cnt <= cnt + 1'b1;  
15    end  
16  
17    // 状态编码 (独热码)  
18    parameter S0 = 4'b0001, S1 = 4'b0010, S2 = 4'b0100, S3 = 4'b1000;
```

```

19  reg [3:0] current_state, next_state;
20
21  // 状态转移同步时序逻辑
22  always @(posedge clk or posedge rst) begin
23      if (rst) current_state <= S0;
24      else current_state <= next_state;
25  end
26
27  // 状态转移组合逻辑
28  always @(*) begin
29      case (current_state)
30          S0: next_state = timer_1s ? S1 : S0;
31          S1: next_state = timer_1s ? S2 : S1;
32          S2: next_state = timer_1s ? S3 : S2;
33          S3: next_state = timer_1s ? S0 : S3;
34          default: next_state = S0;
35      endcase
36  end
37
38  // 输出逻辑
39  always @(posedge clk or posedge rst) begin
40      if (rst) led <= 4'b0001;
41      else led <= current_state;
42  end
43
44  endmodule

```

26. 脉冲边缘检测与计数器及其 Testbench 编写 (15 分)

```
1 // 26-(1) 设计模块
2 module pulse_detect(
3     input clk,
4     input rst,
5     input A,
6     output rise,
7     output reg [15:0] cnt
8 );
9     reg a_d1, a_d2;
10
11     // 打两拍消除亚稳态并用于边沿检测
12     always @(posedge clk or posedge rst) begin
13         if (rst) begin
14             a_d1 <= 1'b0;
15             a_d2 <= 1'b0;
16         end else begin
17             a_d1 <= A;
18             a_d2 <= a_d1;
19         end
20     end
21
22     // 产生单脉冲上升沿指示
23     assign rise = a_d1 && !a_d2;
24
25     // 计数逻辑
26     always @(posedge clk or posedge rst) begin
27         if (rst) begin
28             cnt <= 16'd0;
29         end else if (rise) begin
30             cnt <= cnt + 1'b1;
31         end
32     end
33
34 endmodule
```

```
1 // 26-(2) Testbench 模块
2 `timescale 1ns/1ps
3
4 module pulse_detect_tb;
5     reg clk;
6     reg rst;
7     reg A;
8     wire rise;
```

```

9      wire [15:0] cnt;
10
11     // 例化被测设计
12     pulse_detect uut (
13         .clk(clk),
14         .rst(rst),
15         .A(A),
16         .rise(rise),
17         .cnt(cnt)
18     );
19
20     // 生成时钟 (50MHz)
21     always #10 clk = ~clk;
22
23     initial begin
24         // 初始状态
25         clk = 0;
26         rst = 1;
27         A = 0;
28         #40;
29
30         // 撤销复位
31         rst = 0;
32         #40;
33
34         // 模拟缓慢变化的脉冲 1
35         A = 1; #100;
36         A = 0; #200;
37
38         // 模拟缓慢变化的脉冲 2
39         A = 1; #150;
40         A = 0; #100;
41
42         // 停止仿真
43         $finish;
44     end
45
46 endmodule

```